



Facultad de Ingeniería:

**Abstracción de Software y Manejo de Archivos:
Transpilador Estático de JS a Rust**

Curso:

Sistemas Operativos

Carrera:

Ingeniería Civil en Informática

Integrantes:

Vicente Álvarez

Profesores:

Fabio Sáez / Cristian Olivares-Rodríguez

Índice

1. Introducción	2
1.1. Objetivos de la Solución	2
1.2. Descripción del problema	2
2. Contexto Teórico y Relación con el Sistema Operativo	3
2.1. Código Fuente: JavaScript Original	3
2.2. Implementación del Transpilador en Rust	4
3. Diseño del Algoritmo	5
4. Análisis de Complejidad	5
5. Resultados y Ejecución	6
6. Conclusión	7

1. Introducción

En este informe se comenta el desarrollo de un programa a nivel de usuario que interactúa con el sistema operativo a través de operaciones de **Entrada y Salida (E/S)** para manipular archivos y flujos de texto. Específicamente, se construyó un transpilador (traductor de código fuente a código fuente) escrito en Rust, capaz de leer un archivo de JavaScript (`.js`), procesarlo línea por línea, y emitir el código fuente equivalente en Rust (`.rs`) a través de la salida estándar (`stdout`).

Es importante destacar que este código solo actúa como un traductor, a diferencia de un intérprete o una Máquina Virtual, el programa **no ejecuta** la lógica del código, sino que se limita a traducir la sintaxis y mostrar el nuevo código, delegando la responsabilidad de compilación y ejecución al usuario leyendo el texto de salida.

1.1. Objetivos de la Solución

1. **Gestión de Archivos (Abstracción del S.O.):** Utilizar las llamadas al sistema nativas de la librería estándar de Rust (`std::fs`) para acceder a un archivo almacenado en el disco y cargarlo en la memoria principal para su procesamiento.
2. **Procesamiento de Cadenas y Flujos de Salida:** Analizar el archivo leído para identificar palabras clave de JavaScript (`function`, `const`, `console.log`), reemplazarlas por sus equivalentes en Rust (`fn`, `let`, `println!`) e imprimir el resultado interactuando con la salida estándar del sistema operativo.

1.2. Descripción del problema

El lenguaje JavaScript es notoriamente conocido por ser un lenguaje interpretado lento, ejecutado históricamente en navegadores y hoy en día a través de entornos como NodeJS donde suele ser menos eficiente en consumo de recursos en comparación a lenguajes compilados a nivel de sistema como Rust. El problema abordado es cómo leer un bloque de código básico de JS y transformarlo a Rust utilizando operaciones de E/S, soportando un set limitado de instrucciones: creación de variables, creación de funciones, `console.log` y sumas.

2. Contexto Teórico y Relación con el Sistema Operativo

El problema se modela considerando el manejo de memoria, la entrada/salida y la gestión de procesos del sistema operativo.

- **El Proceso:** Cuando se ejecuta el comando `cargo run`, el sistema operativo asigna memoria RAM y crea un hilo de ejecución principal en modo usuario para el programa transpilador.
- **Salida (Consola):** Al utilizar `println!`, el programa vuelve a solicitar al S.O. que tome las cadenas procesadas en la RAM y las escriba en el búfer de salida estándar (`stdout`), mostrando el código final en la terminal del usuario, cosas de compiladores, donde no nos tenemos que preocupar de las interacciones duras del SO.

2.1. Código Fuente: JavaScript Original

El archivo de entrada (`codex.js`) representa un entorno no tipado que nuestro sistema leerá y convertirá:

Listing 1: Archivo JS de entrada

```
function sum_values(v_1, v_2) {  
    return v_1 + v_2;  
}  
  
const a = 10;  
const b = 20;  
  
const result = sum_values(a, b);  
console.log(result);
```

2.2. Implementación del Transpilador en Rust

El código principal en Rust se estructuró para manejar los argumentos de la línea de comandos (CLI) y procesar la lectura en un bloque seguro, gestionando la posible falla de I/O mediante el tipado Result de Rust.

Listing 2: Transpilador principal en Rust

```
use std::{env, fs, process};

fn phraser(codex: String) {
    println!("fn_main()_{{");
    for line in codex.lines() {
        let rust_line = converter(line);
        if rust_line.trim().is_empty() {
            println!();
        } else {
            println!("_{{_{{", rust_line);
        }
    }
    println!("}}");
}

fn converter(line: &str) -> String {
    let mut out = line.to_string();
    if out.contains("function") {
        out = out.replace("function", "fn")
            .replace(",", ":_i32,")
            .replace("_{{", ":_i32)_->_i32_{{");
    }
    if out.contains("const") {
        out = out.replace("const", "let").replace(
            "_=_", ":_i32_=_");
    }
    if out.contains("console.log(") {
        out = out.replace("console.log(", "
            println!(\"{}_{{\",_{{");
    }
    out
}

fn main() {
    let args: Vec<String> = env::args().collect();
    let filename: String;

    if args.len() < 2 {
        filename = String::from("codex.js");
    } else {
        filename = args[1].clone();
    }

    let js_code = match fs::read_to_string(&filename)
    {
```

```

Ok(content) => content,
Err(e) => {
    eprintln!("No se pudo leer el
        archivo '{}': {}", filename, e
    );
    process::exit(1);
}
};

phraser(js_code);
}

```

3. Diseño del Algoritmo

Para estructurar el funcionamiento lógico de nuestro procesador de texto, dividimos el programa en módulos, aprovechando el paso de variables.

Función/Módulo	Estructuras Clave	Descripción del Comportamiento
<code>main</code>	<code>Vec<String>, fs</code>	Interfaz directa con el S.O. Obtiene argumentos, busca la ruta del archivo y delega la lectura de E/S. Si el archivo no existe, finaliza el proceso ordenadamente (<code>exit(1)</code>).
<code>phraser</code>	<code>String.lines()</code>	Actúa como el motor del ciclo principal. Envuelve el resultado final en la función <code>main</code> requerida por el compilador de Rust e itera el archivo hacia <code>stdout</code> .
<code>converter</code>	<code>String.replace()</code>	Aplica reglas heurísticas de transformación. Muta las referencias de <code>function</code> a <code>fn</code> y tipa dinámicamente las variables a <code>i32</code> .

4. Resultados y Ejecución

El programa fue compilado y ejecutado utilizando el gestor cargo. En la ejecución, el proceso solicitó el archivo `codex.js` al S.O., realizó las mutaciones sintácticas y emitió el resultado mediante la salida estándar.

Listing 3: Salida de la terminal al ejecutar el Transpilador

```
D:\Users\Lithium\Desktop\University\Sistemas operativos\Tarea\RVM>cargo
run
Compiling RVM v0.1.0
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.67s
Running `target\debug\RVM.exe`
fn main() {
    fn sum_values(v_1: i32, v_2: i32) -> i32 {
        return v_1 + v_2;
    }

    let a: i32 = 10;
    let b: i32 = 20;

    let result: i32 = sum_values(a, b);
    println!("{}", result);
}
```

Como se observa, el programa finalizó exitosamente e imprimió en consola el código fuente transpilado y válido para Rust. **Es vital notar que el código de la suma matemática no se ejecutó en ningún momento durante este proceso.** El programa actuó exclusivamente como un manipulador de texto: leyó un archivo, reemplazó caracteres, y escribió un nuevo flujo de texto en la terminal.

5. Conclusión

El desarrollo de este transpilador básico permitió observar en la práctica conceptos de la arquitectura de Sistemas Operativos, específicamente el manejo de la Entrada/Salida (E/S) y la diferencia entre procesar texto y ejecutar código.

En primer lugar, se evidenció el rol del S.O. como **proveedor de abstracciones**. Nuestro programa en Rust no tuvo que calcular en qué bloque físico del disco se encontraba el archivo `codex.js`; la llamada a la librería estándar (`std::fs`) encapsuló las peticiones al hardware en modo Kernel, demostrando cómo el sistema operativo oculta la complejidad del hardware al programador.

Este documento fue asistido por gemini 3.8 flash, originalmente un .docx y pasado a latex con el siguiente prompt: Convierte este pdf a latex manteniendo la estructura anterior de este archivo: [Documento .tex de un informe de Programación Avanzada: https://drive.google.com/file/d/1fRfKTWYfDFn0IHZG9uExUsDj04P0g3_F/view?usp=drive_link]